

UNIT II - Fundamentals of Prompt Engineering

Definition and significance of Prompt Engineering, Types of prompts: Zero-shot, One-shot, Few-shot, Techniques for effective prompt design, Prompt templates and chaining, Prompt tuning and parameter-efficient tuning, evaluating prompt performance, Use of APIs for testing prompts (Open AI, Cohere, Anthropic), Best practices and prompt libraries.

Definition and significance of Prompt Engineering,

What is Prompt Engineering - Meaning, Working, Techniques

Prompt engineering is the process of creating clear and effective prompts that guide AI models to generate accurate responses. It mainly focuses on writing smart prompts for text-based AI tasks, especially NLP, to help the user and the model produce the required output.

Prompt Engineering Importance

- More specific formats of input as prompts help in better interpretability of the requirements for a task.
- Specific prompts with a detailed explanation of the requirements mean output matches more with the desired one.
- Better results for NLP tasks, through prompts also mean a better-trained model for future tasks.

What are Prompts?

Prompts are short pieces of text that are used to provide context and guidance to machine learning models. When talking about the specific text AI tasks, also called NLP tasks, these prompts are useful in generating relevant outputs which are as close to the expected output itself. Precisely, these prompts help in generating accurate responses by:

- Adding on some additional guidance for the model.
- Not generalizing a prompt too much.
- Make sure the information added is not too much as that can confuse the model.
- Making the user intent and purpose clear for the model to generate content in the relevant context only.

How Prompt Engineering Works?

Imagine you're instructing a very talented but inexperienced assistant. You want them to complete a task effectively, so you need to provide clear instructions. Prompt engineering is similar - it's about crafting the right instructions, called prompts, to get the desired results from a large language model (LLM).

Working of Prompt Engineering Involves:

- **Crafting the Prompt:** You design a prompt that specifies what you want the LLM to do. This can be a question, a statement, or even an example. The wording, phrasing, and context you include all play a role in guiding the LLM's response.
- **Understanding the LLM:** Different prompts work better with different LLMs. Some techniques involve giving the LLM minimal instructions (zero-shot prompting), while others provide more context or examples (few-shot prompting).
- **Refining the Prompt:** It's often a trial-and-error process. You might need to tweak the prompt based on the LLM's output to get the kind of response you're looking for.

Applications of Prompt Engineering

Prompt engineering is used most heavily in text-based modeling, especially NLP. It adds context, meaning, and relevance to prompts, helping models generate better outputs. Some key applications include:

- **Language Translation:** It is the process of translating a piece of text from one language to another using relevant language models. Relevant prompts carefully engineering with information like the required script, dialect, and other features of source and target text can help in better response from the model.
- **Question Answering Chatbots:** A Q/A bot is one of the most popular NLP categories to work on these days. It is used by institutional websites, and shopping sites among many others. Prompts on which an AI chatbot Model is trained can largely affect the kind of response a bot generates. An example of what critical information one can add in a prompt can be adding the intent and context of the query so that the bot is not confused in generating relevant answers.
- **Text Generation:** Such a task can have a multitude of applications and hence it again becomes critical to understand the exact dimension of the user's query. The text is generated for what purpose can largely change the tone, vocabulary as well as formation of the text.

Types of Prompts: Zero-shot, One-shot, and Few-shot Prompting

Prompting techniques determine how much guidance or how many examples are provided to a Large Language Model (LLM) before it performs a task.

1. Zero-Shot Prompting

Definition

Zero-shot prompting is a technique where the model is given only the task or question without any examples. The model relies entirely on its pre-trained knowledge to generate the response.

Structure

Instruction → **Response**

Example

Prompt

Translate the following sentence into French:

"Good morning."

Output

Bonjour.

Another Example

Prompt

What is Machine Learning?

Output

Machine Learning is a branch of Artificial Intelligence that enables computers to learn from data and make predictions without being explicitly programmed.

Advantages

- Simple and easy to use.
- Requires no training examples.
- Fast prompt creation.
- Suitable for common tasks.

Limitations

- May produce inconsistent responses.
- Less effective for complex or specialized tasks.
- Depends heavily on the model's prior knowledge.

Applications

- Question answering
- Translation
- Summarization
- General information retrieval

2. One-Shot Prompting

Definition

One-shot prompting provides the model with **one example** before asking it to perform the task. The example helps the model understand the expected format or style.

Structure

One Example → New Input → Response

Example

Prompt

Example:

Input: Apple

Category: Fruit

Now classify:

Input: Carrot

Category:

Output

Vegetable

Another Example

Prompt

Example:

Question: What is the capital of India?

Answer: New Delhi

Question: What is the capital of Japan?

Answer:

Output

Tokyo

Advantages

- Helps the model understand the expected format.
- Improves response accuracy compared to zero-shot.
- Easy to create.

Limitations

- One example may not cover all cases.

- Performance depends on the quality of the example.

Applications

- Text classification
- Sentiment analysis
- Data formatting
- Simple pattern recognition

3. Few-Shot Prompting

Definition

Few-shot prompting provides **multiple examples (typically 2–5 or more)** before asking the model to perform the task. These examples help the model recognize patterns and generate more accurate responses.

Structure

Multiple Examples → New Input → Response

Example

Prompt

Example 1:

Input: Apple

Category: Fruit

Example 2:

Input: Carrot

Category: Vegetable

Example 3:

Input: Rose

Category: Flower

Now classify:

Input: Mango

Category:

Output

Fruit

Another Example

Prompt

Positive: I love this movie.

Positive: The food was delicious.

Negative: The service was terrible.

Sentence:

The hotel room was very clean.

Sentiment:

Output

Positive

Advantages

- Higher accuracy than zero-shot and one-shot.

- Better understanding of patterns.
- Suitable for complex tasks.
- Produces more consistent results.

Limitations

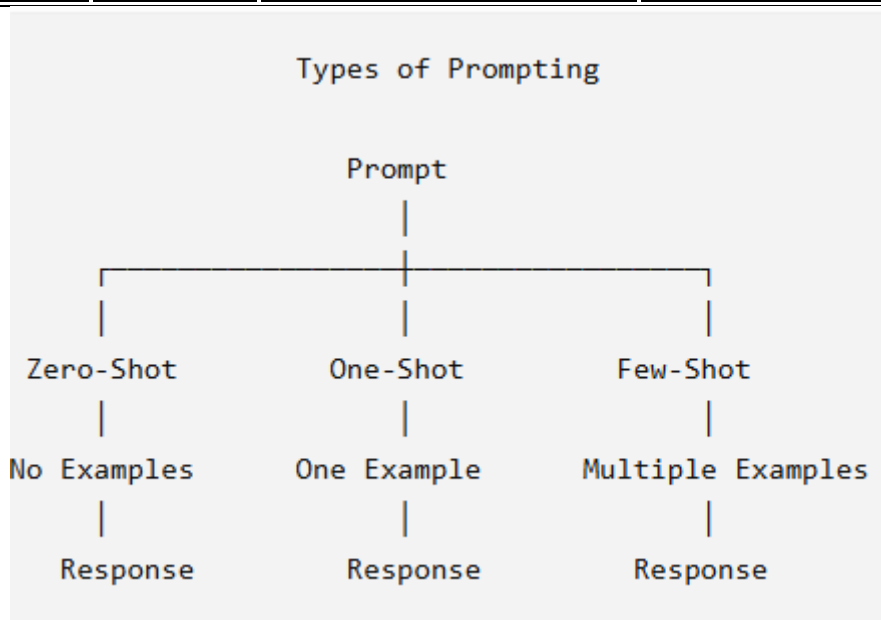
- Requires more tokens.
- Longer prompts increase API cost.
- Carefully chosen examples are needed.

Applications

- Classification
- Named Entity Recognition (NER)
- Information extraction
- Code generation
- Domain-specific tasks

Comparison Table

Feature	Zero-Shot	One-Shot	Few-Shot
Number of Examples	0	1	2 or more
Ease of Use	Very Easy	Easy	Moderate
Accuracy	Moderate	Good	High
Prompt Length	Short	Medium	Long
Cost	Low	Medium	Higher
Best For	General tasks	Simple formatting and classification	Complex reasoning and specialized tasks



Real-World Example

Task: Identify the sentiment of a review.

Zero-Shot

Determine the sentiment:

"The movie was amazing."

Output:

Positive

One-Shot

Example:

"I hated the food." → Negative

Now classify:

"The movie was amazing."

Output:

Positive

Few-Shot

Example 1:

"I loved the music." → Positive

Example 2:

"The service was poor." → Negative

Example 3:

"The product is excellent." → Positive

Now classify:

"The movie was amazing."

Output:

Positive

Prompt templates and chaining

Prompt Templates

Prompt templates are reusable formats for instructing LLMs using placeholders to insert variable information. They are essential for consistency, efficiency and scalability in AI workflows allowing structured, error-free interactions and easy optimization.

Prompt Templates

Purpose of Prompt Templates

Reasons for using prompt templates are:

1. **Standardize Instructions:** Templates provide a consistent way to instruct LLMs, reducing ambiguity in responses.
2. **Enable Reuse Across Tasks:** Once created, templates can be reused for similar tasks, saving time and effort.
3. **Ensure Consistent Outputs:** Using templates helps maintain uniformity in LLM responses, improving reliability.
4. **Reduce Errors and Improve Efficiency:** Structured prompts minimize mistakes and streamline workflow processes for better productivity.

Types of Prompt Templates

Various types of Prompt Templates are:

1. **String Prompt Template:** Simple text-based templates with placeholders for variables.
2. **Chat Prompt Template:** Designed for chat style interactions, supporting multiple roles and messages.
3. **Few Shot Prompt Template:** Includes example inputs and outputs to guide the LLM's behavior for better accuracy.

Using Variables, Rendering and Advanced Customization

Some of the ways to enhance prompt templates are:

1. **Defining and Using Variables:** Specify placeholders in templates and substitute them with dynamic values for flexible prompts.
2. **Formatting and Rendering:** Fill in variables and structure the prompt correctly before sending it to the LLM to ensure consistent outputs.
3. **Custom Templates:** Create templates tailored to specific tasks or domains for higher accuracy.
4. **Logic Based Templates:** Add conditional statements or branching to handle different scenarios dynamically.
5. **Enhanced Control:** Combining variables, rendering and custom logic gives fine grained control over LLM outputs and workflows.

Technique	Prompt template	Description
1. Role Playing	"You are an expert in {field} known for {key adjective}. Help me {task}."	Telling the AI to act as a famous expert or celebrity
2. Style Unbundling	"Describe the key elements of {expert}'s style/skill in bullet points." "Do {task} in the following style: {style}."	Describe what you like about a style rather than copying directly
3. Emotion Prompting	"Help me {task}. Please make sure {attribute}. This task is very important for my career."	Use emotional blackmail and persuasion with the AI
4. Few-Shot Learning	"Here are some examples of {task}. Generate a {task} for {new context}."	Adding examples of the completed task to the prompt

Simple Implementation of Prompt Template

[Example: Student Information Generator](#)

Problem Statement:

Create a Prompt Template that asks the AI to generate basic information about a student in **JSON format**.

Step 1: Install Required Libraries

Install the required Python packages using pip.

```
pip install langchain
pip install langchain-openai
pip install python-dotenv
```

Explanation

- **langchain** – Framework for building LLM applications.
- **langchain-openai** – Connects LangChain with OpenAI models.
- **python-dotenv** – Loads environment variables (API key) securely.

Step 2: Import Modules

```
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from langchain.chains import LLMChain
import os
import json
from dotenv import load_dotenv
```

Explanation

- PromptTemplate → Creates reusable prompt templates.
- ChatOpenAI → Connects to the OpenAI model.
- LLMChain → Executes the prompt with the model.
- json → Converts the AI output into a Python dictionary.
- dotenv → Loads the API key from a .env file.

Step 3: Setup Environment

Create a **.env** file in your project folder.

```
OPENAI_API_KEY=your_openai_api_key
```

Load the API key:

```
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
```

Explanation

The API key is stored securely in the .env file instead of writing it directly in the code.

Step 4: Define Prompt Template

```
template = ""
```

Generate the following student information in JSON format.

Student Name: {name}

Course: {course}

Return only JSON.

Example:

```
{
  "Name": "",
  "Course": "",
  "Skills": [],
  "Career": ""
}
```

```
prompt = PromptTemplate(
    input_variables=["name", "course"],
    template=template
)
```

Explanation

The placeholders:

- {name}
- {course}

will be replaced with actual values.

Step 5: Inspect the Filled Prompt

```
filled_prompt = prompt.format(
    name="Rahul",
    course="Computer Science"
)
```

```
print(filled_prompt)
```

Output

Generate the following student information in JSON format.

Student Name: Rahul

Course: Computer Science

Return only JSON.

Example:

```
{
    "Name": "",
    "Course": "",
    "Skills": [],
    "Career": ""
}
```

Explanation

The placeholders have been replaced with actual values.

Step 6: Initialize the LLM

```
llm = ChatOpenAI(
    model="gpt-3.5-turbo",
    temperature=0
)
```

Explanation

- model → Specifies the AI model.
- temperature=0 → Produces more consistent and deterministic responses.

Step 7: Create an LLMChain

```
chain = LLMChain(  
    llm=llm,  
    prompt=prompt  
)
```

Explanation

The LLMChain combines the prompt template and the language model.

Step 8: Run the Chain

```
result = chain.run(  
    name="Rahul",  
    course="Computer Science"  
)  
print(result)
```

Sample Output

```
{  
  "Name": "Rahul",  
  "Course": "Computer Science",  
  "Skills": [  
    "Programming",  
    "Problem Solving",  
    "Communication"  
  ],  
  "Career": "Software Engineer"  
}
```

Explanation

The AI generates student information in JSON format.

Step 9: Extract and Parse JSON

```
student = json.loads(result)
```

Explanation

The JSON string returned by the AI is converted into a Python dictionary.

Step 10: Display Parsed Results

```
print("Name :", student["Name"])  
print("Course :", student["Course"])  
print("Skills :", student["Skills"])  
print("Career :", student["Career"])
```

Output

Name : Rahul

Course : Computer Science

Skills : ['Programming', 'Problem Solving', 'Communication']

Career : Software Engineer

Complete Program

```
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from langchain.chains import LLMChain
from dotenv import load_dotenv
import os
import json

# Step 1 & 2
load_dotenv()

# Step 3
api_key = os.getenv("OPENAI_API_KEY")

# Step 4
template = """
Generate the following student information in JSON format.

Student Name: {name}
Course: {course}

Return only JSON.

Example:
{
  "Name": "",
  "Course": "",
  "Skills": [],
  "Career": ""
}
"""

prompt = PromptTemplate(
    input_variables=["name", "course"],
    template=template
)

# Step 5
```

```

print(prompt.format(name="Rahul", course="Computer Science"))

# Step 6
llm = ChatOpenAI(
    model="gpt-3.5-turbo",
    temperature=0
)

# Step 7
chain = LLMChain(llm=llm, prompt=prompt)

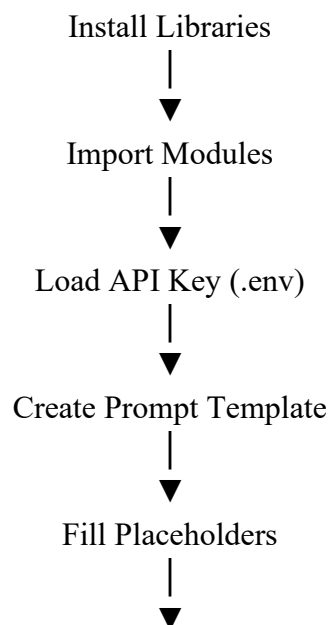
# Step 8
result = chain.run(
    name="Rahul",
    course="Computer Science"
)

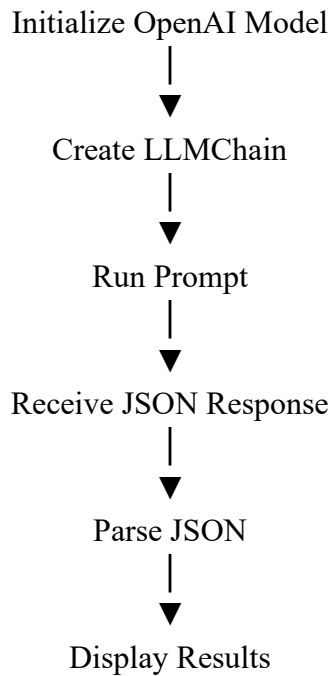
# Step 9
student = json.loads(result)

# Step 10
print("\nStudent Details")
print("-----")
print("Name :", student["Name"])
print("Course :", student["Course"])
print("Skills :", student["Skills"])
print("Career :", student["Career"])

```

Flow Diagram Prompt Template





Applications

Some of the applications of prompt templates are:

1. **Summarization:** Templates help LLMs consistently condense long texts into clear and coherent summaries.
2. **Question Answering:** Structured prompts guide the model to provide accurate and context-aware answers.
3. **Content Generation:** Templates ensure consistent tone and style when generating articles, blogs or marketing copy.
4. **AI Agents and Workflows:** They allow agents to execute multi-step tasks reliably with predefined instructions.
5. **Translation and Localization:** Templates can standardize input-output patterns for translating or adapting content across languages.
6. **Data Extraction:** Prompt templates guide LLMs to extract structured information from unstructured text efficiently.

Benefits

Some of the benefits of prompt templates are:

1. **Reusability:** Templates can be applied to multiple tasks, saving time and reducing repetitive work.
2. **Maintainability:** Structured prompts are easier to update and manage as workflows evolve.
3. **Reduced Errors:** Consistent use of templates minimizes mistakes like variable mismatches or ambiguous instructions.
4. **Efficiency:** Templates streamline workflows by reducing manual effort and improving output consistency.
5. **Scalability:** They make it easier to scale AI applications across multiple tasks or teams without losing quality.
6. **Better Control:** Templates provide predictable behavior, giving developers more control over LLM outputs.

Challenges

Some of the challenges of prompt templates are:

1. **Variable Mismatches:** Incorrect or inconsistent placeholders can lead to errors or unexpected outputs.
2. **Overfitting to Examples:** Few-shot templates may bias the model if the examples are too specific or narrow.
3. **Clarity of Instructions:** Vague prompts can produce inconsistent or irrelevant outputs.
4. **Complexity Management:** Overly complex templates can confuse the LLM and reduce performance.

Prompt Chaining

Prompt chaining is a technique in artificial intelligence especially with large language models (LLMs) where the output of one prompt is used as the input for the next, creating a sequential flow of information and reasoning. This approach allows complex tasks to be broken down into smaller, more manageable steps, guiding the AI through a structured process to achieve more accurate, coherent, and contextually rich results.

How Prompt Chaining Works

1. **Initial Prompt:** You start by giving the AI an initial prompt describing the first step or aspect of a complex task.
2. **First Output:** The model generates a response based on this prompt.
3. **Evaluation and Next Prompt:** The output is evaluated either by a human or an automated system. Based on this, a new prompt is crafted often refining or building upon the previous output.
4. **Chaining:** This process repeats with each new prompt incorporating context or results from the previous step. The chain continues until the desired output is achieved.

Types of Prompt Chaining

- **Sequential Chaining:** Prompts are linked in a straightforward, linear order. Each step depends on the previous one, ideal for tasks like multi-stage writing (outline → draft → edit).
- **Conditional Chaining:** Branches the prompt sequence based on the AI's output. For example, if sentiment analysis returns "positive," the next prompt might ask for positive highlights; if "negative," it might ask for suggested improvements.
- **Looping Chaining :** Iterates a prompt over a collection of items and is useful for batch processing like summarizing multiple documents one by one.

Why Use Prompt Chaining?

- **Breaks Down Complexity :** Decomposes intricate tasks into smaller subtasks, allowing the model to focus on one aspect at a time.
- **Improves Accuracy :** Guides the model's reasoning through intermediate steps, increasing the relevance and precision of the final output.
- **Enhances Explain ability:** Each step is explicit, making it easier to trace how the model arrived at its conclusions.
- **Maintains Context:** By feeding outputs forward, the model retains and builds on context throughout the process, which is especially helpful for long or multistage tasks.

- **Context Length Limitations:** LLMs have a maximum input length, so prompt chaining helps manage tasks that would otherwise exceed these limits by splitting them into smaller, sequential prompts.
- **Workflow Automation:** Prompt chaining can automate multi-step business processes making AI-driven workflows more reliable and customizable.

Prompt Chaining vs. Chain of Thought

Let's see a quick difference between Prompt Chaining vs. [Chain of Thought](#) as they both re quyuite similar:

- **Prompt chaining** breaks a task into distinct, sequential prompts each with its own focus.
- **Chain of Thought** involves a single prompt that encourages the model to reason through multiple steps in one go.

Aspect	Prompt Chaining	Chain of Thought Prompting
Process	Multiple prompts in sequence each tackling a single subtask	One prompt with the model reasoning step by step internally
Structure	Modular, allows iterative refinement after each prompt	Unified, holistic reasoning in a single response
Flexibility	High and easy to adjust or correct individual steps	Lower changes require reworking the whole prompt
Best for	Workflows needing review, iterative learning, content creation	Logic puzzles, math, scenarios needing transparent reasoning
Error Handling	Errors can be addressed at each step	Errors require revisiting the entire response
AI Autonomy	Depends on user or system intervention between steps	More autonomous and self-directed reasoning

What is Prompt Tuning?

Prompt tuning is a technique that involves modifying the input to a pre-trained language model rather than altering the model's parameters. Instead of fine-tuning the entire model, prompt tuning focuses on designing task-specific "prompts" or instructions that guide the model to produce the desired output.

A **prompt** is essentially a structured input that provides context or instructions to the model. For example, instead of directly asking the model to classify a sentence as positive or negative, you might prepend the sentence with a prompt like: **"Classify the sentiment of the following sentence: [sentence]."**

Prompt tuning involves freezing the pre-trained model's parameters and optimizing only a small set of additional parameters, often referred to as "soft prompts." These soft prompts are learned during training and serve as a bridge between the model's pre-trained knowledge and the specific task at hand.

How Does Prompt Tuning Work?

1. **Pre-trained Model:** Start with a pre-trained language model, such as GPT-3 or BERT. The model's parameters remain frozen during the process.
2. **Task-Specific Prompts:** Design a prompt template that includes placeholders for the input data and task-specific instructions. For example:
 - "Translate the following English sentence into French: [English sentence]."
 - "Summarize the following text in one sentence: [text]."
3. **Soft Prompts:** Instead of using fixed, hand-crafted prompts, prompt tuning introduces **learnable soft prompts**. These are continuous vectors that are optimized during training to guide the model toward the desired behavior. Soft prompts are typically initialized randomly and then fine-tuned using gradient descent.
4. **Training:** During training, only the soft prompts are updated, while the rest of the model remains unchanged. This significantly reduces the computational cost compared to full fine-tuning.
5. **Inference:** At inference time, the learned soft prompts are prepended to the input, and the model generates predictions based on the combined input.

Prompt Tuning VS Fine-Tuning VS Prompt Engineering

Aspect	Prompt Tuning	Fine-Tuning	Prompt Engineering
Goal	Improve output by adjusting input prompts.	Adapt the model to a specific task/domain by training.	Craft optimal inputs to guide the model's response.

Aspect	Prompt Tuning	Fine-Tuning	Prompt Engineering
Model Modification	Small learnable parameters or embeddings.	Model weights are updated through training.	No modification to the model itself.
Data Required	May use a small amount of additional data for tuning.	Requires a labeled dataset for the target task.	No new data is needed, just refined prompts.
Use Case	Refining responses with minimal changes to the model.	Specializing a model for a specific task or domain.	Enhancing the model's ability to generate specific responses without retraining.
Complexity	Moderate to low.	High (requires training).	Low to moderate.

When to Use Each:

- **Prompt Tuning** is helpful when you want to **optimize the model's performance on specific prompts** without requiring major retraining. It's suitable for improving accuracy on particular queries or optimizing output generation.
- **Fine-Tuning** is useful when you need the model to handle a **new task** or be specialized in a **specific domain** (e.g., medical text, financial data). Fine-tuning is ideal when you have a substantial amount of labeled data for the target task.
- **Prompt Engineering** is best when you need **immediate control over model responses** and want to craft effective inputs. It's suitable when fine-tuning is not feasible and when you need **optimal outputs** for specific queries without changing the underlying model.

Advantages of Prompt Tuning

1. **Efficiency** : One of the most significant advantages of prompt tuning is its efficiency. Since only a small number of parameters (the soft prompts) are updated, the computational and memory requirements are much lower than those of full fine-tuning. This makes prompt tuning particularly appealing for large models with billions of parameters.
2. **Scalability** : Prompt tuning scales well with model size. Research has shown that as the size of the pre-trained model increases, the performance gap between prompt tuning and full fine-tuning narrows. In some cases, prompt tuning can achieve comparable or even better results than full fine-tuning, especially for very large models.
3. **Task Generalization** : By leveraging the pre-trained model's knowledge, prompt tuning can generalize better across tasks. The model retains its original capabilities while adapting to new tasks through the use of prompts.

4. **Reduced Overfitting** : Since prompt tuning modifies only a small subset of parameters, it is less prone to overfitting, especially when working with limited training data.
5. **Flexibility** : Prompt tuning allows for rapid experimentation with different tasks and prompts without the need to retrain the entire model. This flexibility is particularly valuable in scenarios where multiple tasks need to be addressed simultaneously.

Challenges and Limitations

While prompt tuning offers many advantages, it is not without its challenges:

1. **Prompt Design** : Crafting effective prompts can be non-trivial. The quality of the prompt can significantly impact the model's performance, and designing optimal prompts often requires domain expertise and experimentation.
2. **Interpretability** : Soft prompts are continuous vectors that lack interpretability. Unlike hand-crafted prompts, it is difficult to understand what specific information the soft prompts are encoding.
3. **Task Complexity** : While prompt tuning works well for many tasks, it may struggle with highly complex or multi-step reasoning tasks that require deeper modifications to the model's architecture.
4. **Resource Constraints** : Although prompt tuning is more efficient than full fine-tuning, it still requires access to large pre-trained models, which may not be feasible for all users due to hardware or budget constraints.

Applications of Prompt Tuning

Prompt tuning has found applications across a wide range of NLP tasks, including:

1. **Text Classification** : Prompt tuning can be used to classify text into categories such as sentiment, topic, or intent.
2. **Question Answering** : By providing prompts that guide the model to extract relevant information, prompt tuning can improve performance on question-answering tasks.
3. **Machine Translation** : Prompt tuning can help adapt pre-trained models for translation tasks by incorporating task-specific instructions.
4. **Summarization** : Prompt tuning can be used to generate concise summaries of long documents by guiding the model to focus on key points.
5. **Few-Shot Learning** : Prompt tuning is particularly effective in few-shot learning scenarios, where the model is given only a small number of examples for a new task. By leveraging the pre-trained model's knowledge, prompt tuning can achieve strong performance even with limited data.

Future of Prompt Tuning

As language models continue to grow in size and complexity, techniques like prompt tuning are likely to play an increasingly important role in making these models more accessible and practical. Researchers are actively exploring ways to improve prompt tuning, such as:

- **Automated Prompt Design** : Developing algorithms to automatically generate effective prompts, reducing the need for manual intervention.
- **Hybrid Approaches** : Combining prompt tuning with other parameter-efficient fine-tuning methods, such as adapter modules or LoRA (Low-Rank Adaptation), to further enhance performance.
- **Cross-Task Generalization** : Investigating how prompt tuning can be extended to handle multiple tasks simultaneously, enabling models to perform a wide range of functions without the need for separate fine-tuning.

By focusing on task-specific prompts rather than modifying the entire model, prompt tuning enables users to harness the power of large language models with minimal computational overhead.

Parameter-Efficient Tuning (often called PEFT) is a technique that adapts massive pre-trained models to specific tasks by updating only a tiny fraction of their parameters while keeping the base model frozen. It drastically reduces computational costs, memory usage, and storage requirements compared to full fine-tuning. [1, 2, 3]

Because it only changes a very small subset of weights, PEFT also helps prevent "catastrophic forgetting," allowing the model to retain its broad, foundational knowledge. [1, 2, 3]

Popular PEFT Techniques

- **LoRA (Low-Rank Adaptation):** The most widely used method. It approximates weight updates using low-rank matrices, significantly reducing the number of trainable parameters without adding latency during inference. [1, 2]
- **Prompt Tuning / Prefix Tuning:** Prepends trainable "soft" virtual vectors to the model's inputs or internal layers to guide its behavior. [1, 2]
- **Adapters:** Inserts small, learnable, bottleneck neural network layers between the existing layers of the pre-trained model. [1, 2]

Why PEFT is Widely Used

- **Accessibility:** Allows you to fine-tune massive Large Language Models (LLMs) on consumer-grade hardware or laptops.
- **Modularity:** You can train different lightweight adapters for different tasks (e.g., one for sentiment analysis and one for translation) and swap them easily using the base model.
- **Speed:** Training time and convergence rates are heavily improved. [1, 2, 3, 4]

Full Fine-Tuning vs PEFT

When we compare full fine-tuning with parameter-efficient fine-tuning (PEFT), the differences become clear:

Attribute	Full Fine-Tuning	PEFT (Parameter-Efficient Fine-Tuning)
Parameters Updated	Updates every parameter of the model (billions of weights).	Updates only a small subset or adds small modules; base model stays frozen.
Compute Requirement	Needs very high compute (multi-GPU / TPU).	Can run on a single GPU or modest hardware.
Storage Requirement	Stores a full model for each task; heavy storage usage.	Stores only small adapter weights; base model reused.

Attribute	Full Fine-Tuning	PEFT (Parameter-Efficient Fine-Tuning)
Performance	Strong results but expensive and less scalable.	Almost same performance but much cheaper and scalable.
Practicality	Difficult in low-resource setups; fits large labs.	Practical for edge devices, startups, universities, research groups.

Applications

- Fine-tuning models to run on devices with limited memory or computing power such as mobile phones, IoT devices or embedded systems.
- Using one shared base model with separate lightweight adapters for different tasks. This avoids storing a full copy of the model for each task.
- Creating task-specific or user-specific adapters so organizations and individuals can have custom models without retraining everything.

Limitation

- In some cases, full fine-tuning still delivers slightly higher accuracy, especially for very complex tasks that require deep adaptation.
- It includes techniques like LoRA, Adapters, Prefix-Tuning and BitFit. Each has its strengths but also limitations. Choosing the right one for a specific task is not always straightforward.
- Some PEFT models may perform well on the training dataset but struggle to generalize to different domains or unseen data.
- When we use one model for many tasks, we often end up with many small adapters. Keeping track of them and integrating them efficiently can become complex.

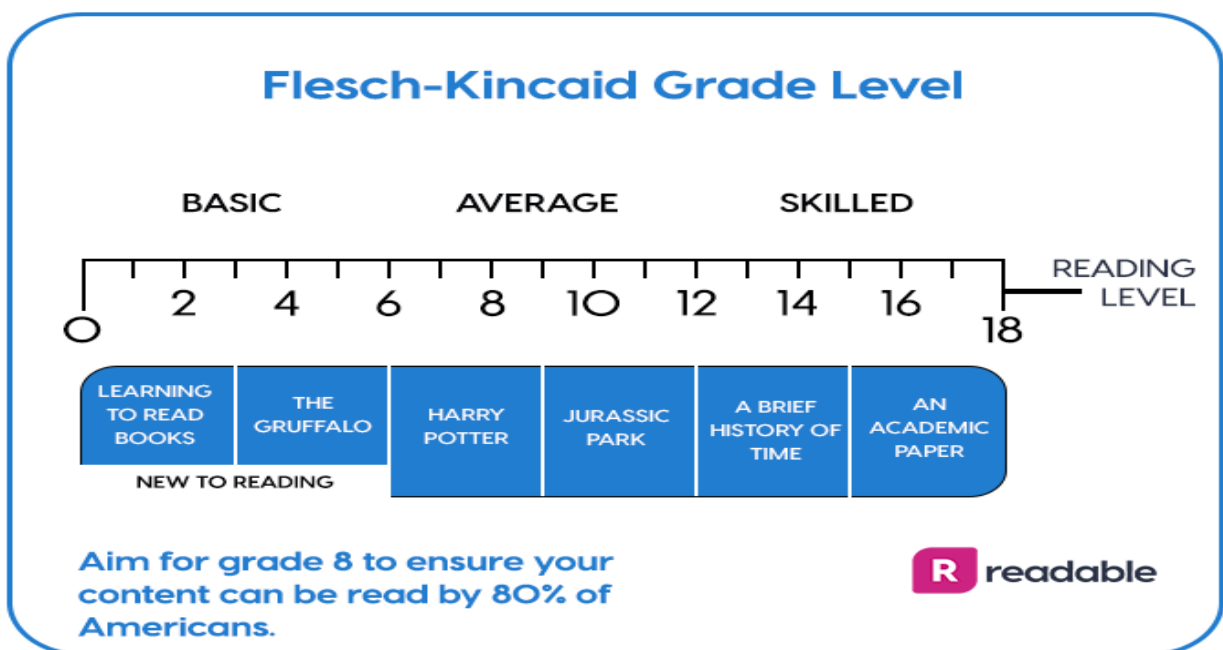
Key Metrics for Evaluating Prompt Effectiveness

Evaluating prompt effectiveness requires a clear set of criteria to judge how well a prompt performs and whether it meets the needs of both the model and the end user. These metrics provide insight into different aspects of prompt quality, from output accuracy to overall user satisfaction.

1. **Relevance** measures how closely the model's output aligns with the user's original intent in the query. This is particularly important when dealing with complex or nuanced prompts where slight deviations can lead to misunderstood outputs. Relevance can be measured by comparing the output with a predefined desired outcome, often using similarity scores (e.g., cosine similarity for embeddings) or manual evaluations.
2. **Accuracy** is the degree to which the output is factually correct or aligns with known information, especially critical in domains requiring precision, such as medical information, legal guidance, or technical support. Accuracy can be evaluated by

comparing the output against trusted reference data or external knowledge bases. Metrics like **BLEU**, **ROUGE**, or **F1 scores** are commonly used for automated scoring.

3. **Consistency** assesses whether the model provides reproducible and similar responses when the same prompt is input multiple times or across different instances. Consistency can be measured by running the same prompt multiple times and comparing results for similarity. A high degree of similarity indicates consistency, whereas large variations may suggest issues with the prompt or the model's tuning.
4. **Efficiency** evaluates the speed and resource usage of the model when generating outputs, which becomes increasingly important in real-time applications where latency and computational load are factors. Efficiency can be tracked using response time (latency) and by monitoring computational metrics like CPU/GPU usage and memory load during prompt processing.
5. **Readability & Coherence**
Readability and coherence measure the clarity, logical flow, and overall intelligibility of the output. This is particularly relevant in applications where user interaction or content consumption is involved. Readability can be measured using readability formulas (like Flesch-Kincaid) or manually evaluating sentence structure, grammar, and flow. Coherence might require subjective evaluation, often by comparing the response with known coherent texts or gathering user feedback.

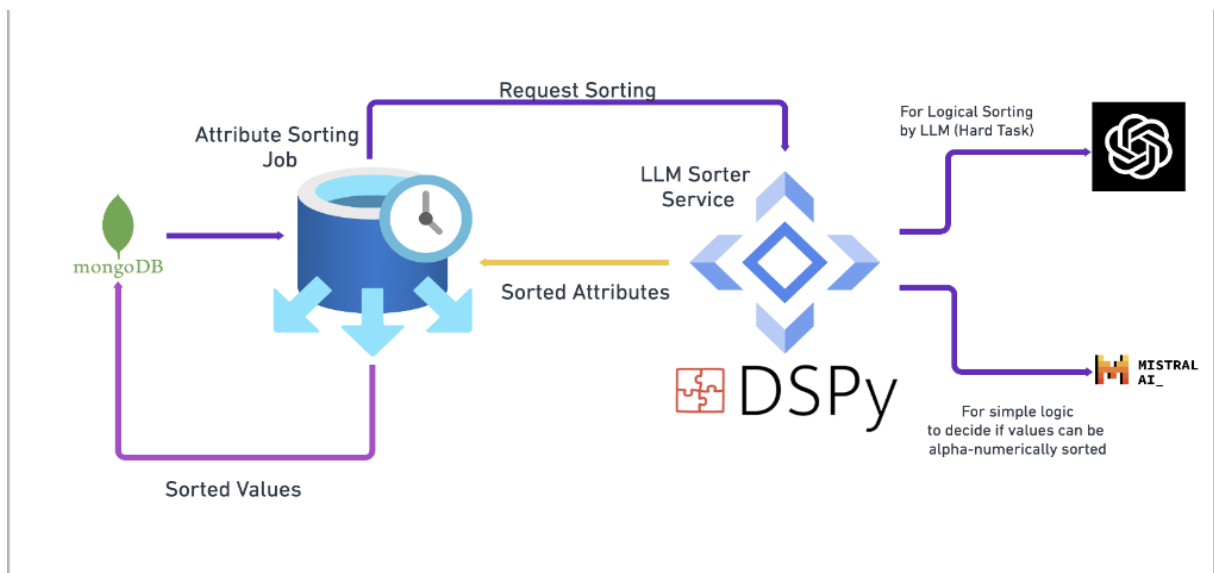


Flesch-Kincaid: [Image source](#)

6. User Satisfaction Score

User Satisfaction Score reflects how satisfied users are with the output, usually gathered through feedback mechanisms like surveys or scoring systems embedded in the application. You can use tools like Portkey to get [weighted feedback](#) from customers on any request you serve, at any stage in your app.

Evaluation Libraries



Evaluating prompt performance

Evaluating prompt performance means systematically measuring how well an AI model responds to your instructions. Key steps include setting metrics like **relevance** and **accuracy**, building a **test dataset** of real-world inputs, and scoring outputs manually or using **LLM-as-a-judge** frameworks.

To establish a solid evaluation pipeline, focus on these five core pillars:

1. Define Target Metrics

Determine what success looks like for your specific use case. Common evaluation dimensions include:

- **Relevance:** Does the response directly address the user's query without unnecessary filler?
- **Accuracy & Grounded ness:** Is the information factually correct and aligned with your source data?
- **Coherence & Readability:** Are instructions clear, logical, and easy to follow?
- **Task Completion:** Did the model follow all constraints (e.g., length limits, formatting, forbidden words)?
- **Latency & Cost:** Are the tokens used efficiently, and is the response time acceptable for your users

2. Build a Representative Test Dataset

Do not rely on a single, manual test case. Gather 20-100 diverse inputs that represent edge cases, varied user intents, and common production traffic. This ensures your prompts do not suffer from quality degradation when scaling.

3. Choose Your Evaluation Method

You can choose to evaluate your test set using one or a combination of these methods:

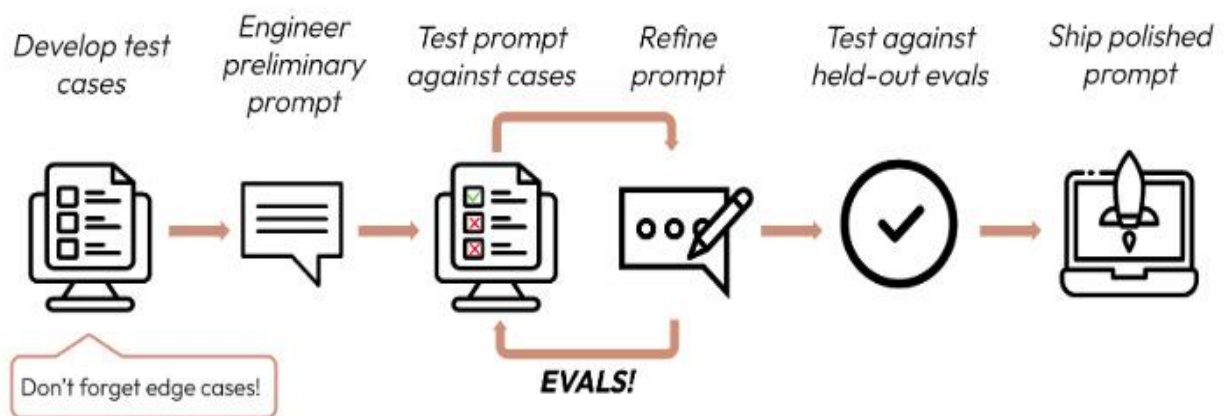
- **Human Evaluation:** Manually score responses based on a predefined rubric (e.g., 1-5 stars for clarity). It is the most reliable method but time-consuming.

- **LLM-as-a-Judge:** Use a more powerful model (like GPT-4 or Claude 3.5) to automatically evaluate and score prompt generations based on criteria you provide
 - **Automated Metrics:** For strict outputs, use programmatic scoring like **Exact Match**, **Precision/Recall/F1**, or similarity scores (BLEU/ROUGE).
- 4. A/B Test and Backtest**
- Never deploy a new prompt blind. Run your new prompt variant against your test dataset and compare it to the older version's responses. You can use **back testing** to pass historical, production inputs through your new prompt to measure exactly how the output behavior changes.

5. Monitor and Iterate

Once live, continuously collect user feedback (like upvotes or task success signals). Identify where the model fails or frustrates users, use those real-world inputs to update your test dataset, and refine the prompt instructions to fix regressions.

evaluating prompt performance



Use of APIs for testing prompts

Use of APIs for Testing Prompts

APIs (Application Programming Interfaces) allow developers and researchers to interact with Large Language Models (LLMs) programmatically. They are widely used to test, compare, and evaluate prompts across different AI models such as OpenAI, Anthropic, and Cohere.

Why Use APIs for Prompt Testing?

- Automate prompt evaluation.
- Compare responses from multiple AI models.
- Measure response quality, accuracy, and consistency.
- Test different prompt variations quickly.
- Integrate AI into applications and workflows.

General Workflow

Step 1: Obtain an API Key

- Create an account with the AI provider.
- Generate an API key.
- Store the key securely.

Step 2: Install Required Library

`pip install openai`

Similarly,

`pip install anthropic`

`pip install cohere`

Step 3: Write the Prompt

Example Prompt:

Explain Machine Learning in simple words.

Step 4: Send Request through API

The API sends the prompt to the model.

Step 5: Receive Response

The model generates an answer which can be evaluated.

1. OpenAI API Example

Install

`pip install openai`

Python Code

```
from openai import OpenAI
```

```
client = OpenAI(api_key="YOUR_API_KEY")
```

```
response = client.responses.create(  
    model="gpt-5.5",  
    input="Explain Artificial Intelligence in simple words."  
)
```

```
print(response.output_text)
```

Sample Output

Artificial Intelligence (AI) is the ability of computers to perform tasks that normally require human intelligence, such as learning, reasoning, and decision-making.

2. Anthropic API Example

Install

`pip install anthropic`

Python Code

```
import anthropic

client = anthropic.Anthropic(
    api_key="YOUR_API_KEY"
)

message = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=200,
    messages=[
        {
            "role": "user",
            "content": "Explain Artificial Intelligence."
        }
    ]
)

print(message.content[0].text)
```

Sample Output

Artificial Intelligence is a branch of computer science that enables machines to learn, solve problems, and make decisions like humans.

3. Cohere API Example

Install

```
pip install cohere
```

Python Code

```
import cohere

co = cohere.Client("YOUR_API_KEY")

response = co.chat(
    model="command-r-plus",
    message="Explain Artificial Intelligence."
)

print(response.text)
```

Sample Output

Artificial Intelligence refers to computer systems that can perform tasks requiring human intelligence, such as learning, reasoning, and problem-solving.

Comparing Responses from Different APIs

API	Model Example	Response Speed	Strengths
OpenAI	GPT-5.5	Fast	Excellent reasoning, coding, and general knowledge

API	Model Example	Response Speed	Strengths
Anthropic	Claude Sonnet	Fast	Long-context understanding, analysis, and document processing
Cohere	Command R+	Fast	Retrieval-augmented generation (RAG), enterprise applications

Example Prompt Testing

Prompt:

Summarize the importance of renewable energy in 50 words.

Run the same prompt using:

- OpenAI API
- Anthropic API
- Cohere API

Compare:

- Accuracy
- Completeness
- Grammar
- Word Limit
- Hallucinations
- Response Time

Evaluation Metrics

Metric	Description
Accuracy	Is the information correct?
Relevance	Does the response answer the prompt?
Completeness	Are all required points included?
Fluency	Is the language clear and natural?
Consistency	Are repeated responses similar?
Latency	Time taken to generate a response

Metric	Description
Cost	API usage cost per request
Safety	Does the response avoid harmful or inappropriate content?

Advantages of API-Based Prompt Testing

- Automated testing of prompts
- Easy comparison across multiple models
- Faster experimentation
- Supports integration into applications
- Enables performance measurement and benchmarking
- Scalable for large numbers of prompt evaluations

Limitations

- API usage incurs costs.
- Requires internet connectivity.
- Different providers have different rate limits.
- Model outputs may vary over time as providers update their models.

Best Practices

- Use clear and specific prompts.
- Keep temperature and other generation settings consistent when comparing models.
- Test prompts with multiple inputs and edge cases.
- Record latency, cost, and output quality.
- Evaluate responses using predefined metrics rather than subjective impressions.
- Iterate and refine prompts based on evaluation results.

Simple Example for Students

Prompt:

"What is Cloud Computing?"

OpenAI Output: Focuses on a clear explanation with examples.

Anthropic Output: Provides a detailed conceptual explanation.

Cohere Output: Gives a concise, enterprise-oriented explanation.

By comparing these responses, students can determine which prompt and model combination best meets their requirements.

Best Practices and Prompt Libraries

Prompt engineering is the process of designing effective prompts to obtain accurate, relevant, and reliable responses from Large Language Models (LLMs). Following best practices and using prompt libraries helps improve AI performance while reducing trial and error.

Part A: Best Practices for Prompt Engineering

1. Write Clear and Specific Prompts

Provide precise instructions instead of vague questions.

Poor Prompt

Tell me about AI.

Better Prompt

Explain Artificial Intelligence in 100 words with three real-world applications.

Benefit: Produces more focused and relevant responses.

2. Provide Context

Giving background information helps the model understand the task better.

Example

You are a Computer Science professor.

Explain Cloud Computing to first-year B.Tech students using simple language.

Benefit: Responses are tailored to the intended audience.

3. Define the Output Format

Specify how you want the answer to be presented.

Example

List five advantages of IoT in a table.

Output

Advantage	Description
Automation	Reduces manual work

Advantage	Description
Monitoring	Enables real-time tracking
Efficiency	Improves productivity
Cost Saving	Lowers operational costs
Decision Making	Supports data-driven decisions

4. Use Step-by-Step Instructions

Break complex tasks into smaller steps.

Example

Explain Machine Learning by:

1. Defining it
2. Explaining its types
3. Giving two applications
4. Mentioning one advantage

5. Use Examples (Few-Shot Prompting)

Provide examples to guide the model.

Example

Input: Apple
Category: Fruit

Input: Carrot
Category: Vegetable

Input: Mango
Category:

Output

Fruit

6. Assign a Role

Specify the role the AI should play.

Example

Act as a Cyber Security Expert and explain phishing attacks.

7. Specify Constraints

Mention limits such as word count, tone, or audience.

Example

Summarize this article in exactly 100 words.

8. Iterate and Refine Prompts

Improve prompts based on the responses received.

Version 1

Explain AI.

Version 2

Explain AI in simple language with real-life examples suitable for Class 10 students.

9. Verify Important Information

For critical tasks, validate AI-generated responses using trusted sources or expert review.

10. Test Multiple Prompt Variations

Compare different prompts to identify which produces the best results.

Part B: Prompt Libraries

A prompt library is a collection of reusable prompt templates for common tasks such as summarization, coding, translation, content generation, and data analysis.

Advantages

- Saves time
- Ensures consistency
- Reduces prompt-writing effort
- Improves productivity
- Enables prompt reuse across projects

Popular Prompt Libraries

1. Open AI Prompt Examples

Provides sample prompts for learning and experimentation.

Typical categories:

- Text generation
- Summarization
- Translation

- Code generation
- Question answering

2. LangChain Prompt Templates

LangChain provides reusable prompt templates that allow variables to be inserted dynamically.

Example

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate.from_template(
    "Explain {topic} in simple words."
)

prompt = template.invoke({"topic": "Machine Learning"})
print(prompt.text)
```

Output

Explain Machine Learning in simple words.

3. Microsoft Guidance

A prompt programming framework that helps create structured and reusable prompts for LLM applications.

Used for:

- Controlled generation
- Prompt workflows
- Structured outputs

4. PromptSource

An open-source collection of prompts designed for NLP datasets and benchmarking.

Common uses:

- Research
- Model evaluation
- Benchmark testing

5. FlowGPT

A community platform where users share prompts for:

- Chatbots
- Coding
- Marketing
- Education
- Creative writing

6. Awesome ChatGPT Prompts

A curated GitHub repository containing prompts for many domains.

Categories include:

- Programming
- Education
- Business
- Resume writing
- Data analysis
- Interview preparation

Example of a Reusable Prompt Template

Role:
You are an experienced teacher.

Task:
Explain the given topic.

Topic:
Artificial Intelligence

Audience:
First-year B.Tech students

Output Format:
Bullet points with one real-world example.

Word Limit:
150 words

Comparison of Prompt Libraries

Prompt Library	Purpose	Best For
OpenAI Prompt Examples	Learning prompt design	Beginners
LangChain Prompt Templates	Dynamic prompt creation	AI application development
Microsoft Guidance	Structured prompt programming	Advanced LLM workflows
PromptSource	Research and benchmarking	NLP researchers
FlowGPT	Community-shared prompts	Prompt inspiration

Prompt Library	Purpose	Best For
Awesome ChatGPT Prompts	Large collection of reusable prompts	General-purpose use

Best Practices Summary

- Write clear and specific prompts.
- Provide sufficient context.
- Define the desired output format.
- Break complex tasks into steps.
- Use examples when appropriate.
- Assign a role to the AI.
- Specify constraints (length, tone, audience).
- Test and refine prompts iteratively.
- Validate important outputs.
- Maintain a reusable prompt library for consistency and efficiency.

Simple Definition:

Best Practices: Techniques used to design effective prompts that improve the quality, accuracy, and consistency of AI-generated responses.

Prompt Library: A collection of reusable prompt templates that helps users efficiently perform common AI tasks with consistent results.